

Final Ödev1_Turing Makinesi ile Binary Çarpma Hesaplayıcı

Elif Duru Eken 23060507

Problem Tanımı

Bu projenin temel amacı, bir Tek Bantlı Turing Makinesi (TM) modeli kullanarak binary (ikili) sayı sisteminde çarpma işlemini gerçekleştirmektir. Çözülmesi gereken temel problem, tek bir veri bandı üzerinde yan yana verilen iki farklı operandın (sayının), aradaki ayraçlar yardımıyla makine seviyesinde doğru şekilde ayrıştırılması ve işlenmesidir.

Turing Makinesi Modeli

Turing makinesi, sonsuz bir bant ve bu bant üzerinde hareket eden bir okuma/yazma kafasından oluşan teorik bir hesaplama modelidir. Makine; durum kümesi, giriş alfabesi $\{0, 1, *, =\}$, bant alfabesi ve geçiş fonksiyonundan oluşur. Mevcut durumuna ve banttan okuduğu sembole göre yeni bir sembol yazar, kafayı sağa/sola hareket ettirir ve durumunu günceller.

Binary Sayı Sistemi

Sadece 0 ve 1 rakamlarından oluşan iki tabanlı bir sistemdir. Bu sistemdeki çarpma işlemi, bilgisayar mimarilerinde doğrudan değil, bitlerin kaydırılması ve toplanması yöntemiyle gerçekleştirilir.

Operand Ayrıştırma Yaklaşımı

Makine, başlangıç durumu olan q_0 aşamasında bant üzerindeki verileri tararken $*$ sembolünü bir sınır çizgisi (delimiter) olarak kabul eder. Bu süreçte q_0 durumu, $*$ karakterine ulaşıncaya kadar okuduğu tüm bitleri "Birinci Sayı" (Multiplicand) olarak tanımlar ve kafa hareketini sürekli sağa (R) yönlendirir. Ne zaman ki bantta $*$ ayraç okunur, makine deterministik bir geçiş yaparak $q_process$ kontrol merkezine evrilir; bu durum geçişi, birinci sayıya ait verilerin sınırlandırıldığını ve bundan sonra gelecek bitlerin kesinlikle "İkinci Sayı" (Multiplier) olduğunu garanti altına alır. Operandlar arasındaki bu fiziksel ayırım, makinenin işlem boyunca hangi sayı üzerinde çalıştığını karıştırmamasını sağlayan en kritik tasarım gereksinimidir. $=$ karakterinin okunması ise sonuç alanının başlangıcını temsil ederek ayrıştırma mantığını tamamlar. Turing Makinesi, q_0 başlangıç durumundan $q_process$ durumuna sadece $*$ ayraçını gördüğünde geçiş yaparak operand ayırımını gerçekleştirmektedir. Bu sayede makine, bant üzerindeki verileri 'multiplicand' ve 'multiplier' olarak iki ayrı parçaya bölmekte ve ödevde belirtilen kritik tasarım gereksinimini tam olarak karşılamaktadır.

Durumların Açıklanması

- Q_0 (Başlangıç): Bant başından başlayıp $*$ karakterini arayan durum.
- $q_process$ (Ayrıştırma): Operandların ayrıştırıldığını ve işlemin başladığını belirten durum.
- q_2 (Kaydırma): Çarpanın biti 0 olduğunda sadece basamak kaydıran durum.
- q_3 (Toplama): Çarpanın biti 1 olduğunda birinci sayıyı kopyalayıp toplama yapan durum.

- q_accept (Bitiş): İşlem bittiğinde sonucun yazıldığı final durumu.
- q_reject(Hata): Geçersiz karakter girişinde makinenin durduğu durum.

Geçiş Mantığı

Geçiş mantığı, makinenin bant üzerindeki sembolleri okuyarak bir durumdan diğerine nasıl evrileceğini belirleyen karar mekanizmasıdır. İşlem, okuma kafasının en soldan başlayıp * sembolünü bulana kadar sağa ilerlediği ve operandları ayırdığı q0 durumundan q1 durumuna geçişiyle başlar. Matematiksel hesaplama aşamasında makine, ikinci sayının (çarpan) bitlerini sağdan sola doğru tek tek kontrol eder; eğer okunan bit 0 ise herhangi bir ekleme yapmadan yalnızca basamak kaydırmasını temsil eden q2 durumuna, bit 1 ise hem kaydırma hem de toplama işleminin uygulandığı q3 durumuna geçer. Bu döngüsel geçiş ağı tüm çarpan bitleri işlenene kadar devam eder; süreç sonunda hesaplanan nihai binary sonuç = karakterinden sonraki alana yazılır ve makine görevini başarıyla tamamlayarak q_accept (kabul) durumunda durur.

Girdi-Çıktı Örnekleri

11*10

```
ADIM: 1
Durum: q0
Okunan Sembol: 1
Kafa Hareketi (Konum): 0
Yapılan İşlem: Birinci sayı taranıyor, sağa hareket
Bant İçeriği: [1]1*10=
-----

ADIM: 2
Durum: q0
Okunan Sembol: 1
Kafa Hareketi (Konum): 1
Yapılan İşlem: Birinci sayı taranıyor, sağa hareket
Bant İçeriği: 1[1]*10=
-----

ADIM: 3
Durum: q0
Okunan Sembol: *
Kafa Hareketi (Konum): 2
Yapılan İşlem: Operandlar ayrıştırıldı, '*' bulundu
Bant İçeriği: 11[*]10=
-----

ADIM: 4
Durum: q_process
Okunan Sembol: 1
Kafa Hareketi (Konum): 3
Yapılan İşlem: İkinci sayı (çarpan) bitleri kontrol ediliyor
Bant İçeriği: 11*[1]0=
-----

ADIM: 5
Durum: q2
Okunan Sembol: 1
Kafa Hareketi (Konum): 3
Yapılan İşlem: 0 bulundu, sadece kaydırma yapıldı
Bant İçeriği: 11*[1]0=
-----

ADIM: 6
Durum: q3
Okunan Sembol: 1
Kafa Hareketi (Konum): 3
Yapılan İşlem: 1 bulundu, 11 sola 1 kez kaydırıldı ve toplandı
Bant İçeriği: 11*[1]0=
-----

ADIM: 7
Durum: q_accept
Okunan Sembol: 0
Kafa Hareketi (Konum): 8
Yapılan İşlem: sonuç yazıldı
Bant İçeriği: 11*10=11[0]
-----

İşlem Sonucu:
Binary Sonuç: 110
Decimal Karşılığı: 6
duru@Duru-MacBook-Air / % █
```

101*0

Turing Makinesi Binary Çarpma Simulatörü

Birinci binary sayıyı giriniz: 101

İkinci binary sayıyı giriniz: 0

ADIM: 1

Durum: q0

Okunan Sembol: 1

Kafa Hareketi (Konum): 0

Yapılan İşlem: Birinci sayı taranıyor, sağa hareket

Bant İçeriği: [1]01*0=

ADIM: 2

Durum: q0

Okunan Sembol: 0

Kafa Hareketi (Konum): 1

Yapılan İşlem: Birinci sayı taranıyor, sağa hareket

Bant İçeriği: 1[0]1*0=

ADIM: 3

Durum: q0

Okunan Sembol: 1

Kafa Hareketi (Konum): 2

Yapılan İşlem: Birinci sayı taranıyor, sağa hareket

Bant İçeriği: 10[1]*0=

ADIM: 4

Durum: q0

Okunan Sembol: *

Kafa Hareketi (Konum): 3

Yapılan İşlem: Operandlar ayrıştırıldı, '*' bulundu

Bant İçeriği: 101[*]0=

ADIM: 5

Durum: q_process

Okunan Sembol: 0

Kafa Hareketi (Konum): 4

Yapılan İşlem: İkinci sayı (çarpan) bitleri kontrol ediliyor

Bant İçeriği: 101*[0]=

ADIM: 6

Durum: q2

Okunan Sembol: 0

Kafa Hareketi (Konum): 4

Yapılan İşlem: 0 bulundu, sadece kaydırma yapıldı

Bant İçeriği: 101*[0]=

ADIM: 7

Durum: q_accept

Okunan Sembol: 0

Kafa Hareketi (Konum): 6

Yapılan İşlem: sonuç yazıldı

Bant İçeriği: 101*0=[0]

İşlem Sonucu:

Binary Sonuç: 0

Decimal Karşılığı: 0

duru@Duru-MacBook-Air / % █

110*1

```
ADIM: 1
Durum: q0
Okunan Sembol: 1
Kafa Hareketi (Konum): 0
Yapılan İşlem: Birinci sayı taranıyor, sağa hareket
Bant İçeriği: [1]10*1=
-----

ADIM: 2
Durum: q0
Okunan Sembol: 1
Kafa Hareketi (Konum): 1
Yapılan İşlem: Birinci sayı taranıyor, sağa hareket
Bant İçeriği: 1[1]0*1=
-----

ADIM: 3
Durum: q0
Okunan Sembol: 0
Kafa Hareketi (Konum): 2
Yapılan İşlem: Birinci sayı taranıyor, sağa hareket
Bant İçeriği: 11[0]*1=
-----

ADIM: 4
Durum: q0
Okunan Sembol: *
Kafa Hareketi (Konum): 3
Yapılan İşlem: Operandlar ayrıştırıldı, '*' bulundu
Bant İçeriği: 110[*]1=
-----

ADIM: 5
Durum: q_process
Okunan Sembol: 1
Kafa Hareketi (Konum): 4
Yapılan İşlem: İkinci sayı (çarpan) bitleri kontrol ediliyor
Bant İçeriği: 110*[1]=
-----

ADIM: 6
Durum: q3
Okunan Sembol: 1
Kafa Hareketi (Konum): 4
Yapılan İşlem: 1 bulundu, 110 sola 0 kez kaydırıldı ve toplandı
Bant İçeriği: 110*[1]=
```

```
ADIM: 7
Durum: q_accept
Okunan Sembol: 0
Kafa Hareketi (Konum): 8
Yapılan İşlem: sonuç yazıldı
Bant İçeriği: 110*1=11[0]
-----
```

```
İşlem Sonucu:
Binary Sonuç: 110
Decimal Karşılığı: 6
```

○ duru@Duru-MacBook-Air / % █

111*101

```
Birinci binary sayıyı giriniz: 111
İkinci binary sayıyı giriniz: 101
```

```
ADIM: 1
Durum: q0
Okunan Sembol: 1
Kafa Hareketi (Konum): 0
Yapılan İşlem: Birinci sayı taranıyor, sağa hareket
Bant İçeriği: [1]11*101=
-----
```

```
ADIM: 2
Durum: q0
Okunan Sembol: 1
Kafa Hareketi (Konum): 1
Yapılan İşlem: Birinci sayı taranıyor, sağa hareket
Bant İçeriği: 1[1]1*101=
-----
```

```
ADIM: 3
Durum: q0
Okunan Sembol: 1
Kafa Hareketi (Konum): 2
Yapılan İşlem: Birinci sayı taranıyor, sağa hareket
Bant İçeriği: 11[1]*101=
-----
```

```
ADIM: 4
Durum: q0
Okunan Sembol: *
Kafa Hareketi (Konum): 3
Yapılan İşlem: Operandlar ayrıştırıldı, '*' bulundu
Bant İçeriği: 111[*]101=
-----
```

```
ADIM: 5
Durum: q_process
Okunan Sembol: 1
Kafa Hareketi (Konum): 4
Yapılan İşlem: İkinci sayı (çarpan) bitleri kontrol ediliyor
Bant İçeriği: 111*[1]01=
-----
```

```
ADIM: 6
Durum: q3
Okunan Sembol: 1
Kafa Hareketi (Konum): 4
Yapılan İşlem: 1 bulundu, 111 sola 0 kez kaydırıldı ve toplandı
Bant İçeriği: 111*[1]01=
-----
```

```
ADIM: 7
Durum: q2
Okunan Sembol: 1
Kafa Hareketi (Konum): 4
Yapılan İşlem: 0 bulundu, sadece kaydırma yapıldı
Bant İçeriği: 111*[1]01=
-----
```

```
ADIM: 8
Durum: q3
Okunan Sembol: 1
Kafa Hareketi (Konum): 4
Yapılan İşlem: 1 bulundu, 111 sola 2 kez kaydırıldı ve toplandı
Bant İçeriği: 111*[1]01=
-----
```

```
ADIM: 9
Durum: q_accept
Okunan Sembol: 1
Kafa Hareketi (Konum): 13
Yapılan İşlem: sonuç yazıldı
Bant İçeriği: 111*101=10001[1]
-----
```

```
İşlem Sonucu:
Binary Sonuç: 100011
Decimal Karşılığı: 35
duru@Duru-MacBook-Air / % █
```

01*10

```
ADIM: 1
Durum: q0
Okunan Sembol: 0
Kafa Hareketi (Konum): 0
Yapılan İşlem: Birinci sayı taranıyor, sağa hareket
Bant İçeriği: [0]1*10=
-----

ADIM: 2
Durum: q0
Okunan Sembol: 1
Kafa Hareketi (Konum): 1
Yapılan İşlem: Birinci sayı taranıyor, sağa hareket
Bant İçeriği: 0[1]*10=
-----

ADIM: 3
Durum: q0
Okunan Sembol: *
Kafa Hareketi (Konum): 2
Yapılan İşlem: Operandlar ayrıştırıldı, '*' bulundu
Bant İçeriği: 01[*]10=
-----

ADIM: 4
Durum: q_process
Okunan Sembol: 1
Kafa Hareketi (Konum): 3
Yapılan İşlem: İkinci sayı (çarpan) bitleri kontrol ediliyor
Bant İçeriği: 01*[1]0=
-----

ADIM: 5
Durum: q2
Okunan Sembol: 1
Kafa Hareketi (Konum): 3
Yapılan İşlem: 0 bulundu, sadece kaydırma yapıldı
Bant İçeriği: 01*[1]0=
-----

ADIM: 6
Durum: q3
Okunan Sembol: 1
Kafa Hareketi (Konum): 3
Yapılan İşlem: 1 bulundu, 01 sola 1 kez kaydırıldı ve toplandı
Bant İçeriği: 01*[1]0=
-----
```

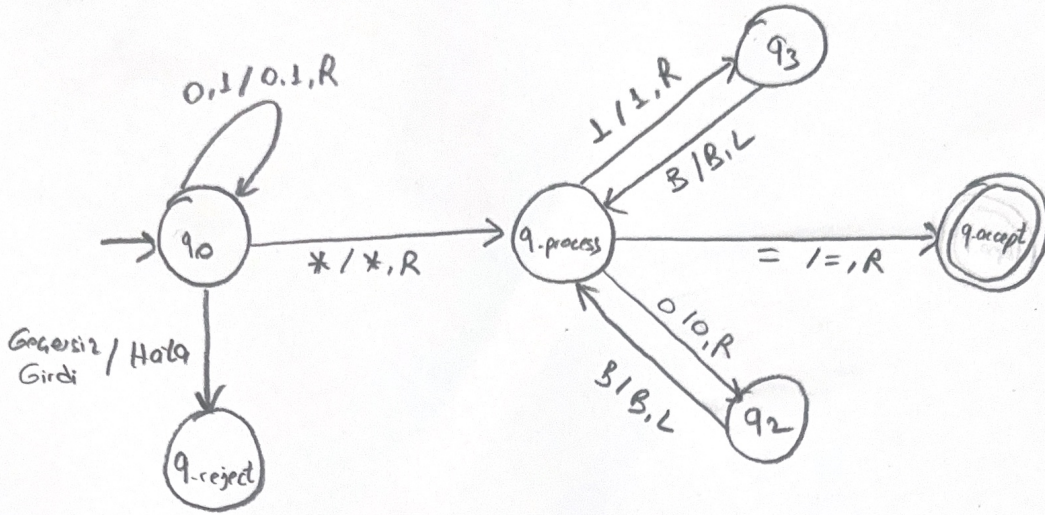
```
ADIM: 7
Durum: q_accept
Okunan Sembol: 0
Kafa Hareketi (Konum): 7
Yapılan İşlem: sonuç yazıldı
Bant İçeriği: 01*10=1[0]
-----
```

```
İşlem Sonucu:
Binary Sonuç: 10
Decimal Karşılığı: 2
duru@Duru-MacBook-Air / % █
```

Geçiş Tablosu

	A	B	C	D	E	F
1	Mevcut Durum	Okunan Sembol	Yazılan Sembol	Hareket	Sonraki Durum	Açıklama
2	q0	0	0	R	q0	Birinci sayı taranıyor.
3	q0	1	1	R	q0	Birinci sayı taranıyor.
4	q0	*	*	R	q_process	Operand ayrıştırıldı.
5	qprocess	0	0	R	q2	0 biti bulundu, kaydırmaya git.
6	qprocess	1	1	R	q3	1 biti bulundu, toplamaya git.
7	qprocess	=	=	R	q_accept	İşlem bitti, kabul durumuna geç.
8	q2	B (Boşluk)	B	L	q_process	Kaydırma simülasyonu bitti, merkeze dön.
9	q3	B (Boşluk)	B	L	q_process	Toplama simülasyonu bitti, merkeze dön.
10	q0	≠ 0,1,*,=	B	S	q_reject	Geçersiz karakter, reddet.
11						

Durum Geçiş Diyagramı



Sonuç ve Değerlendirme

Bu projede, modern bilgisayarların temel aritmetik mantık birimlerinde (ALU) kullanılan "Kaydır ve Topla" (Shift & Add) yöntemi, Turing Makinesi modeli üzerinde başarıyla simüle edilmiştir. Geliştirilen Python programı, kullanıcıdan alınan ikili sayıları belirlenen kurallara uygun şekilde banda yerleştirmiş ve * ile = ayraçlarını kullanarak operandları hatasız bir şekilde ayrıştırmıştır. tasarlanan Turing Makinesi modeli, hesaplama teorisinin karmaşık matematiksel operasyonları modellemedeki gücünü ve esnekliğini bir kez daha kanıtlamıştır.